EXP.NO:	8	BANKER'S ALGORITHM (DEADLOCK AVOIDANCE)
DATE:		

AIM

To implement the Banker's Algorithm for Deadlock Avoidance and determine whether the system is in a safe state by finding a safe sequence.

PROCEDURE

1. Input the number of processes and resource types.
2. Read the Allocation matrix, Maximum matrix, and Available resources.
3. Compute the Need matrix: Need[i][j] = Max[i][j] - Allocation[i][j].
4. Apply the safety algorithm to find a safe sequence.
5. If all processes complete, system is safe; otherwise deadlock may occur.

PROGRAM

```
#include <stdio.h>

int main() {
    int p, r, i, j, k;
    printf("Enter number of processes: "); scanf("%d", &p);
    printf("Enter number of resource types: "); scanf("%d", &r);

    int alloc[p][r], max[p][r], need[p][r];
    int avail[r], work[r], finish[p], safeSeq[p];

    printf("Enter Allocation Matrix:\n");
    for(i=0;i<p;i++) for(j=0;j<r;j++) scanf("%d",&alloc[i][j]);
    printf("Enter Max Matrix:\n");
    for(i=0;i<p;i++) for(j=0;j<r;j++) scanf("%d",&max[i][j]);
    printf("Enter Available Resources:\n");
    for(i=0;i<r;i++) { scanf("%d",&avail[i]); work[i]=avail[i]; }

    for(i=0;i<p;i++) for(j=0;j<r;j++) need[i][j]=max[i][j]-alloc[i][j];
    for(i=0;i<p;i++) finish[i]=0;

    int count=0;
    while(count<p) {
        int found=0;
        for(i=0;i<p;i++) {
            if(finish[i]==0) {
                int possible=1;
                for(j=0;j<r;j++) if(need[i][j]>work[j]) { possible=0; break; }
                if(possible) {
                    for(k=0;k<r;k++) work[k]+=alloc[i][k];
                    safeSeq[count++]=i; finish[i]=1; found=1;
                }
            }
        }
        if(!found) break;
    }
    if(count==p) {
        printf("System is in a SAFE STATE.\n");
        printf("Safe Sequence: ");
        for(i=0;i<p;i++) printf("P%d ",safeSeq[i]);
        printf("\n");
    } else printf("NOT in safe state. Deadlock may occur.\n");
    return 0;
}
```

OUTPUT

```
$ gcc bankers.c
$ ./a.out
Enter number of processes: 5
Enter number of resource types: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Max Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
3 3 2

System is in a SAFE STATE.
Safe Sequence: P1 P3 P4 P0 P2
```

RESULT

The Banker's Algorithm was successfully implemented. The system was determined to be in a safe state and the safe sequence P1 → P3 → P4 → P0 → P2 was found, confirming no deadlock would occur.